

```
# python3 -m pip install tabulate
from tabulate import tabulate
from fractions import Fraction

# Participating students:
# Trisha Bajpai, Tommy Dalessio, Kathleen Reece, Theo Tran

# Constructs a Difference Distribution Table (DDT)
# Input: sbox - dictionary representing an S-box
# Input: l - int number of bits accepted by the S-box
# Output: DDT, represented as a list, with each element of the list being a row in the table
def construct_ddt(sbox: dict, l: int) -> list:
    table_size = 2**l # For l = 3, 2^3 = 8, so 8x8 table
    table = []

    for x_prime in range(table_size):
        y_prime_freq = {i: 0 for i in range(table_size)}
        for x in range(table_size):
            x_star = x ^ x_prime
            y = sbox[x]
            y_star = sbox[x_star]
            y_prime = y ^ y_star
            y_prime_freq[y_prime] += 1
        table.append(list(y_prime_freq.values()))

    return table

# Filters 4-tuples to find "right" pairs for a given differential trail.
#
# Input: tuples - list of (x, x_star, y, y_star) as 6-bit integers
#         predicted_diff: the expected difference at the target bits
#         bit_mask: bitmask selecting which output bits the trail targets
# Output: list of right 4-tuples
def filter_right_tuples(tuples: list) -> list:
    right = []
    for x, x_star, y, y_star in tuples:
        if ((y >> 3) == (y_star >> 3)):
            right.append({
                "x": format(x, "06b"),
                "x_star": format(x_star, "06b"),
                "y": format(y, "06b"),
                "y_star": format(y_star, "06b")
            })
    return right

def format_tuple_table(right_list):
    rows = [[d["x"], d["x_star"], d["y"], d["y_star"]] for d in right_list]
    return tabulate(rows, headers=["x", "x*", "y", "y*"], tablefmt="grid")

# Helper function to get counts for each key guess
def run_counter(right_tuples, sbox_inv, expected_u4_diff):
    counts = {k: 0 for k in range(8)}

    for tup in right_tuples:
        # 1. Get the 3 bits of ciphertext that entered S32
        y_int = int(tup["y"], 2) & 0b111
        ys_int = int(tup["y_star"], 2) & 0b111

        for k in range(8):
            # 2. XOR with our key guess (v = y ^ k)
            v = y_int ^ k
            v_star = ys_int ^ k

            # 3. Use inverse S-box to go back to the S-box input (u)
            u = sbox_inv[v]
            u_star = sbox_inv[v_star]

            # 4. Check if the input difference matches our trail's prediction
            if (u ^ u_star) == expected_u4_diff:
                counts[k] += 1

    return counts

def main():
    # S-Box definition:
    # Input:  0, 1, 2, 3, 4, 5, 6, 7
    # Output: 6, 5, 1, 0, 3, 2, 7, 4
    sbox = {0: 6, 1: 5, 2: 1, 3: 0, 4: 3, 5: 2, 6: 7, 7: 4}
    # Add inverse S-box
    sbox_inv = {v: k for k, v in sbox.items()}

    # Set S-box bit length
    l = 3

    # Construct Difference Distribution Table (DDT)
    ddt_table = construct_ddt(sbox, l)

    # Format the table for output using tabulate
    headers = ["Input Diff/ Output Diff"] + [f"{{i}}" for i in range(2**l)]
    table_string = tabulate(ddt_table, headers=headers, showindex="always", tablefmt="grid")

    # Write output to file
    with open("Project4.txt", "w") as f:
        f.write(f"Project 4: Cryptography MATH 4175\n")
        f.write(f"Participating Students:\n")
        f.write(f"Trisha Bajpai, Tommy Dalessio, Kathleen Reece, Theo Tran\n\n")
        f.write(f"Part 1: Difference Distribution Table (ND(a',b'))\n")
        f.write(f"Rows: Input Difference, Columns: Output Difference\n\n")
        f.write(table_string)

    # Propagation Ratios for the three trails
    # Each ratio is the product of (DDT count / 8) for each active S-box in the trail

    # Trail Tr1 (P6 -> H6): S12(001->001), S22(001->001)
    R1 = (ddt_table[1][1] / 8) * (ddt_table[1][1] / 8)

    # Trail Tr2 (P6 -> H5,H6): S12(001->011), S21(001->001), S22(001->001)
    R2 = (ddt_table[1][3] / 8) * (ddt_table[1][1] / 8) * (ddt_table[1][1] / 8)

    # Trail Tr3 (P6 -> H4,H5,H6): S12(001->011), S21(001->011), S22(001->001)
    R3 = (ddt_table[1][3] / 8) * (ddt_table[1][3] / 8) * (ddt_table[1][1] / 8)

    # Append results to output file
    with open("Project4.txt", "a") as f:
        f.write("\n\nParts 2 & 3: Propagation Ratios\n\n")
        ratio_data = [
            ["Tr1 (P6 -> H6)", f"{Fraction(R1).limit_denominator()}", R1],
            ["Tr2 (P6 -> H5,H6)", f"{Fraction(R2).limit_denominator()}", R2],
            ["Tr3 (P6 -> H4,H5,H6)", f"{Fraction(R3).limit_denominator()}", R3],
        ]
        f.write(tabulate(ratio_data,
            headers=["Trail", "Ratio (fraction)", "Ratio (decimal)"],
            tablefmt="grid"))
```

```
# 4-tuples (x, x*, y, y*) as 6-bit integers
raw_tuples = [
    (0b100111, 0b100110, 0b100100, 0b111110),
    (0b000111, 0b000110, 0b110010, 0b110110),
    (0b001100, 0b001101, 0b111001, 0b100000),
    (0b011000, 0b011001, 0b011101, 0b011111),
    (0b001000, 0b001001, 0b001101, 0b000011),
    (0b011010, 0b011011, 0b101001, 0b101000),
]

# Right tuples are the same for all trails, since all
# have output differences only in the last 3 bits
right_tuples = filter_right_tuples(raw_tuples)

with open("Project4.txt", "a") as f:
    f.write("\n\nPart 4: Filtering for Right 4-Tuples\n\n")

    f.write("All 4-tuples with y^y* values:\n")
    all_rows = [
        [format(x,"06b"), format(xs,"06b"), format(y,"06b"), format(ys,"06b"), format(y^ys,"06b")]
        for x, xs, y, ys in raw_tuples
    ]
    f.write(tabulate(all_rows, headers=["x","x*","y","y*","y^y*"], tablefmt="grid"))

    f.write("\n\nRight 4-tuples:\n")
    f.write(format_tuple_table(right_tuples) if right_tuples else "None found.\n")

# Part 5: Subkey recovery for the last round
sbox_inv = {v: k for k, v in sbox.items()}
EXPECTED_U4_DIFF_TR1 = 0b000001
EXPECTED_U4_DIFF_TR2 = 0b000011
EXPECTED_U4_DIFF_TR3 = 0b000111

# Pass the correct arguments: (tuples, inverse_sbox, target_difference)
c1 = run_counter(right_tuples, sbox_inv, EXPECTED_U4_DIFF_TR1)
c2 = run_counter(right_tuples, sbox_inv, EXPECTED_U4_DIFF_TR2)
c3 = run_counter(right_tuples, sbox_inv, EXPECTED_U4_DIFF_TR3)

with open("Project4.txt", "a") as f:
    f.write("\n\nPart 5: Weighted Cryptanalysis\n")

    trails = [("a", "Tr1", c1), ("b", "Tr2", c2), ("c", "Tr3", c3)]
    for label, name, counts in trails:
        f.write(f"\n({label}) Counter c{label} for Trail {name}:\n")
        rows = [[k, format(k, "03b"), counts[k]] for k in range(8)]
        f.write(tabulate(rows, headers=["Key", "Binary", f"c{label}"], tablefmt="grid") + "\n")

    # (d) Weighted average C = R1*c1 + R2*c2 + R3*c3
    weighted_results = []
    final_scores = {}
    for k in range(8):
        C = (R1 * c1[k]) + (R2 * c2[k]) + (R3 * c3[k])
        final_scores[k] = C
        weighted_results.append([k, format(k, "03b"), c1[k], c2[k], c3[k], f"{C:.6f}"])

    f.write("\n(d) Weighted Average Counts C:\n")
    f.write(tabulate(weighted_results,
        headers=["Key", "Binary", "c1", "c2", "c3", "Count C"],
        tablefmt="grid"))

    # (e) Conclusion
    max_c = max(final_scores.values())
    best_keys = [k for k, v in final_scores.items() if v == max_c]
    f.write(f"\n\n(e) Conclusion:\n")
    f.write(f"Last 3 bits of K4: {'', '.join(format(k, '03b') for k in best_keys)}\n")

# Call main function
if __name__ == "__main__":
    main()
```